

THE ZERO PARADOX

ZP-N: The Constructive Snap

the ε_0 snap from below, choice-free on ordinal notations

Initial: July 2026 | Current: 2.0, July 2026

The constructive companion to ZP-L. The snap-from-below is rebuilt syntactically on ordinal notations (ONote), where it is choice-free — [propext] only. Beside it: a carrier sized to ε_0 whose crossing into Mathlib's Ordinal is one named map with a measured price, and a proof that the generality ZP-L borrows is genuinely non-constructive. Proved sorry-free in Lean 4.

ZP-L derived the snap at ε_0 , and every ε_0 result in its Lean development carries Classical.choice. ZP-N asks where that choice actually lives and whether the snap needs it. Two answers, and they point in opposite directions. The snap's downward structure — the ω -tower climbs without bound, and no ordinal notation is a fixed point of $x \mapsto \omega^x$ — is genuinely constructive, provable with no Axiom of Choice. But the machinery ZP-L reaches for to state that structure semantically is not: comparing arbitrary well-orders implies excluded middle. So the choice is not a mere artifact of how a proof was written. It is the price of a tool stronger than the job requires. Whether ZP-L's ε_0 results could be re-proved without it remains unclassified — no choice-free re-proof of them exists, and nothing here supplies one.

So this layer does two things. It rebuilds the ascent syntactically, where it is choice-free. And it builds a carrier sized to ε_0 — ordinal notations with one point adjoined on top — where the crossing into Mathlib's semantic ordinals is a single named map whose cost can be read off directly. Staying on the notation side is free of choice; crossing is not; and the crossing is one function rather than a diffuse dependency.

The move is to work at the level of ordinal notations (ONote, Cantor-normal-form terms), whose comparison ONote.cmp is choice-free (propext-only) and never passes through repr into the semantic Ordinal type. ε_0 is not itself a notation: the notations name exactly the ordinals below ε_0 , and ε_0 is their limit — the fixed point the notation system cannot reach. So the snap threshold is characterised from below: the tower ascends, and the fixed point (ε_0) is precisely what lies beyond every notation. This layer adds no axiom; it isolates where choice actually enters the ordinal snap.

Section I: The Syntactic Substrate

The Cantor normal form gives each ordinal below ε_0 a finite syntactic name. At the notation level, ω^x is simply the term `oadd x 1 0` (leading exponent x , coefficient 1, empty remainder) — no general ordinal exponentiation is needed. The ω -tower is built by iterating it. Comparison of notations is the purely syntactic ONote.cmp, which returns `lt` / `eq` / `gt` by structural recursion and never evaluates `repr`, so it is choice-free.

Definitions (ConstructiveOrdinals.lean)

`omegaPow x = oadd x 1 0` — the notation for ω^x ; purely syntactic and computable.

`tower 0 = 0`, `tower (n+1) = omegaPow (tower n)` — the ω -tower $(\omega)^{[n]}$ 0, constructive.

Definitions (ConstructiveOrdinals.lean)

ONote.cmp — the syntactic three-way comparison on notations (lt / eq / gt); choice-free (it does not touch repr / Ordinal).

Section II: The Choice-Free Snap from Below

Lemma: exp_lt_term (ConstructiveOrdinals.lean)

For every notation e, coefficient n, and remainder a: $\text{cmp } e \text{ (oadd } e \text{ n a)} = \text{lt}$.

An exponent is strictly below its own term. Proof: structural induction on the exponent, pure syntax — no repr, no Ordinal.

Lean purity: [proptext] only — choice-free, and free even of Quot.sound. ✓

Theorem: omegaPow_no_fixedpoint (ConstructiveOrdinals.lean)

For every notation x: $\text{cmp } x \text{ (omegaPow } x) = \text{lt}$.

No ordinal notation is a fixed point of $x \mapsto \omega^x$: every notation is strictly below its own ω -power, in the choice-free syntactic comparison (holds for all notations, well-formed or not). This is the constructive shadow of “ ϵ_0 is the least fixed point of $x \mapsto \omega^x$, lying beyond every notation.”

Lean purity: [proptext] only — choice-free. ✓

Theorem: tower_strictMono (ConstructiveOrdinals.lean)

For every n: $\text{cmp } (\text{tower } n) \text{ (tower } (n+1)) = \text{lt}$.

The ω -tower is strictly increasing: the snap stages climb without bound below ϵ_0 . Immediate from omegaPow_no_fixedpoint applied at tower n.

Lean purity: [proptext] only — choice-free. ✓

The snap from below is choice-free. All three results depend on [proptext] alone — not Classical.choice, and not even Quot.sound. The ascent of the ω -tower and the absence of any notation fixed point are proved by pure structural recursion on the syntax. This is the sharpest form of the framework’s choice-free-core pattern (see ZeroParadox/AxiomProfile.lean): the snap’s downward structure needs nothing beyond propositional extensionality.

Section III: The Finding and Its Fences

The point of ZP-N is the contrast with ZP-L. There, every ϵ_0 result carries Classical.choice; here, the same downward structure is proved with [proptext] only. It is tempting to conclude that the choice is therefore an artifact of representation. It is not, and the reason is worth stating precisely, because the natural diagnosis is wrong twice over.

First, the choice is not in the type. `#print axioms Ordinal` reports `[propext, Quot.sound]`. It is in the order instance and the operations built on it — `Ordinal.partialOrder`, `instLinearOrder`, `typein`, `omega0`, `nfp`, `deriv`, `epsilon`. Second, and consequently, an axiom footprint on any ε_0 result measures that ambient instance rather than the proof: $a \leq a$ carries `Classical.choice` while $a = a$ does not, for the same element, differing only in whether the statement mentions the order (`order_footprint_le`, `order_footprint_eq`). The instrument cannot answer the question it was being asked.

What it can answer is a different question: is that instance’s classical content real? It is. Merely knowing, of any two well-orders, that one of them embeds in the other — without being handed which — already yields excluded middle (`em_of_wellOrder_comparable`, itself choice-free: the classical content is the hypothesis, not the proof). The hypothesis is a bare disjunction, not a procedure that computes the answer, which makes the implication sharper than it first sounds; it is also the form Mathlib’s `le_total` actually takes. This is a known taboo in constructive mathematics, not a result of this framework: it is Theorem 38(d) of Kraus, Nordvall Forsberg and Xu, stated there in the data form, and their witnesses are the ones used here. So the generality Mathlib’s order provides — comparing any two well-orders — genuinely requires the classical assumption. ε_0 never needed that generality.

Remark: even well-formedness inherits choice (the bridge, made visible)

`tower_NF` — the statement that each tower stage is in normal form — DOES carry `Classical.choice` (`[propext, Classical.choice, Quot.sound]`), because Mathlib’s `NF` predicate is defined through `repr` into `Ordinal`. The snap facts (Section II) do not depend on `NF`, so they stay choice-free; but the fact that even “this notation is well-formed” inherits choice pins the location precisely: choice lives at the syntax→semantics bridge, exactly where `ZP-L` crosses it and `ZP-N` does not.

That diagnosis suggests its own remedy: build a carrier sized to the job. `E0Note` is ordinal notations with a single point adjoined on top, that point standing for ε_0 . Below the top, comparison is the existing syntactic comparator and stays decidable; the map `e0Repr` sends the carrier into Mathlib’s `Ordinal`. Measured, the boundary is priced: the carrier side carries no `Classical.choice` anywhere, and the map carries it at every declaration. Staying constructive is free; crossing costs the classical assumption; and the crossing is one named function.

Scope fences on the carrier — what it is and is not

The fixed point at the top is stipulated, not discovered. Defining the tower operator to fix the adjoined point makes the closure exist by fiat at the added point; only its uniqueness is a theorem. This does not weaken `no_snap_closure`, which says no such closure exists on the notations alone.

`E0Note` is a notation system for $\varepsilon_0 + 1$, not for ε_0 . The standard alternative is a Veblen system where ε_0 is an ordinary term; the top’s honest advantage is that it is minimal and leaves the comparator untouched.

Correctness in Castéran’s sense (`ON_correct`) is not claimed: on raw notations the representation map is not injective, which `e0Repr_not_injective` proves. Restricting to normal forms is the standard repair and is not done here, because that predicate is itself defined through the representation map.

Scope fences on the carrier — what it is and is not

The construction is not new. The carrier is Castéran's generic sum of notation systems (hydra-battles, ON_plus — that file carries his name alone, though the library as a whole is his and Contejean's) instantiated with a one-point right summand. The map is an instance of the same notion his ON_correct defines, not a member of that class: ON_correct has three fields and only the first is established here, as the bullet above records. What is contributed is the measurement.

Scope fence: the snap from below, not minimality

ZP-N proves the snap from below: ε_0 is the fixed point unreachable from within the notation system (no notation is a fixed point of $x \mapsto \omega^x$; the tower climbs without bound). The matching minimality — that ε_0 is the LEAST fixed point of $x \mapsto \omega^x$ — is a separate, harder target: it quantifies over the limit, which no notation names, so it cannot be stated purely syntactically on ONote. That direction remains open; what is proved here is the from-below half, choice-free.

Theorem Summary

Result	Lean source (full path)	Axioms
exp_lt_term	ZeroParadox/Ordinal/ConstructiveOrdinals.lean	propext only
omegaPow_no_fixedpoint	ZeroParadox/Ordinal/ConstructiveOrdinals.lean	propext only
tower_strictMono	ZeroParadox/Ordinal/ConstructiveOrdinals.lean	propext only
tower_NF	ZeroParadox/Ordinal/ConstructiveOrdinals.lean	Classical.choice
tower_cofinal	ZeroParadox/Ordinal/SnapNucleusConstructive.lean	propext only
no_snap_closure	ZeroParadox/Ordinal/SnapNucleusConstructive.lean	propext only
em_of_wellOrder_comparable	ZeroParadox/Ordinal/OrdinalChoiceEssential.lean	propext, Quot.sound
order_footprint_eq	ZeroParadox/Ordinal/OrdinalChoiceEssential.lean	propext, Quot.sound
order_footprint_le	ZeroParadox/Ordinal/OrdinalChoiceEssential.lean	Classical.choice
E0Note (carrier)	ZeroParadox/Ordinal/PricedInterface.lean	no axioms
e0Repr (the crossing)	ZeroParadox/Ordinal/PricedInterface.lean	Classical.choice
e0Repr_not_injective	ZeroParadox/Ordinal/PricedInterface.lean	Classical.choice

Axiom Purity

The snap from below (exp_lt_term, omegaPow_no_fixedpoint, tower_strictMono): [propext] only — choice-free, and free even of Quot.sound. Proved by structural recursion on ONote; repr / Ordinal are never touched.

Well-formedness (tower_NF): [propext, Classical.choice, Quot.sound] — Classical.choice inherited from Mathlib's NF / repr, which passes through the Ordinal type.

Axiom Purity

The taboo (`em_of_wellOrder_comparable`): [`propext`, `Quot.sound`] — choice-free by design, because the classical content is the hypothesis rather than the proof. Its converse (`comparable_of_classical`) carries `Classical.choice`, as it must.

The carrier (`E0Note`, `e0Coe`): no axioms at all; its decidable-order instances [`propext`, `Quot.sound`]. The crossing (`e0Repr` and every lemma about it): [`propext`, `Classical.choice`, `Quot.sound`]. The carrier side carries no choice anywhere.

Together these locate the `Classical.choice` in ZP-L's ε_0 results: it is not in the Ordinal type, and not in the ascent (which is proved choice-free here), but in the order instance the semantic statement passes through — where its classical content is load-bearing. Locating it is not eliminating it: whether those ε_0 results admit a choice-free re-proof is unclassified. Zero sorry. Verified: lake build, July 2026.

End of ZP-N | The Constructive Snap | the snap from below on ordinal notations, choice-free ([`propext`] only): `exp_lt_term`, `omegaPow_no_fixedpoint`, `tower_strictMono` | the classical dependency is in Mathlib's order instance, where comparing arbitrary well-orders implies excluded middle (taboo cited to Kraus, Nordvall Forsberg and Xu) | a carrier sized to ε_0 , with the crossing priced at one named map | minimality (ε_0 the least fixed point) open.